Advanced Web Technologies (AWT)
(2301CS511)

Darshan UNIVERSITY
योग: कर्मसु कौशलम्

Unit-01

Hooks in ReactJS

Prof. Arjun V. Bala
Computer Engineering Department
Darshan Institute of Engineering & Technology, Rajkot
arjun.bala@darshan.ac.in
9624822202

Outline

- ✓ Introduction to hook
- ✓ useState
- ✓ useEffect
- ✓ useContext
- ✓ useActionState
- ✓ useFormStatus
- ✓ Creating Custom Hook in React
- ✓ useDebugValue
- ✓ useCallback
- ✓ useMemo
- ✓ useRef
- ✓ useTransition
- ✓ useDeferedValue
- ✓ useImperativeHandle
- ✓ useLayoutEffect
- ✓ useOptimistic
- ✓ useReducer
- ✓ useSyncExternalStore

Darshan UNIVERSITY

योग: कर्मसु कौशलम्

State Management

- *Hooks* are a new addition in React 16.8.
- Using Hooks you can use state and other React features without writing a class, because of hooks class components are no longer needed.
- Hooks allow us to "hook" into React features such as state and lifecycle methods.
- We can use built-in hooks or create our custom hook if we want.
- If we want to use any of the built-in hooks we need to import them from the react.
- There are 3 rules to use hooks
  - Hooks can only be called inside React function components.
  - Hooks can only be called at the top level of a component.
  - Hooks cannot be conditional
- In this chapter we are going to explore many hooks like

✓ useState	✓ useContext	✓ useLayoutEffect	✓ useRef
✓ useEffect	✓ useDebugValue	✓ useMemo	✓ useSyncExternalState
✓ useActionState	✓ useDeferedValue	✓ useOptimistic	✓ useTransition
✓ useCallback	✓ useInteractiveHandle	✓ useReducer	✓ useFormStatus

“useState” Hook

- The React useState Hook allows us to track state in a function component.
- State generally refers to data or properties that need to be tracking in an application.
- In order to use useState hook we first need to import it from the react.

```
import { useState } from 'react';
```

- In order to initialize the state we can call useState in our function.

```
const [ count , setCount ] = useState(0);
```

- Note: here we are destructuring the returned values from useState.
- useState accepts an initial state and returns two values:
  - The current state.
  - A function that updates the state.
- To read the state we can simply use count variable in our function.
- To update the state we must use the function returned from the useState call

```
setCount(20); // this is valid
count = 20;  // this is not valid
```

“useEffect” Hook

- The useEffect Hook allows you to perform side effects in your components.
- Some examples of side effects are:
  - fetching data
  - directly updating the DOM
  - Timers
  - Etc…
- useEffect accepts two arguments. The second argument is optional.

```
useEffect(<function>, <dependency>)
```

- We can perform side effects in our component specified at the function argument.
- Here notice that this function will be executed each time we render the component.
- There are several ways to control when side effects run using the second argument.

Darshan UNIVERSITY

“useEffect” Hook (Cont.)

- No dependency passed

```
useEffect(() => {
    //Runs on every render
  });
```

- An empty array:

```
useEffect(() => {
    //Runs only on the first render
  }, []);
```

- Dependency passed (dependency can be props, state etc…)

```
useEffect(() => {
    //Runs when there is a change in a or b
  }, [a,b]);
```

“useContext” Hook

- useContext provides easy access to shared state/data from React’s Context API without passing props down manually.

```
import { createContext, useContext } from "react";
const ThemeContext = createContext("light");
function DisplayTheme() {
  const theme = useContext(ThemeContext);
  return <span>Current theme: {theme}</span>;
}
function App() {
  return (
    <ThemeContext.Provider value="dark">
      <DisplayTheme />
    </ThemeContext.Provider>
  );
}
```

“useActionState” Hook

- useActionState helps manage state that depends on form submissions or server actions. Available in React Canary/experimental and supports React Server Components.

```
import { useActionState } from "react";
function increment(prevState, formData) {
  console.log(formData.get("FirstName"));
  return prevState + 1;
}
function StatefulForm() {
  const [state, formAction] = useActionState(increment, 0);
  return (
    <form>
      {state}
      <input type="text" name="FirstName" />
      <button formAction={formAction}>Increment</button>
    </form>
  );
}
```

“useFormStatus” Hook

- useFormStatus is a Hook that gives you status information of the last form submission.

```
import { useFormStatus } from "react-dom";

function SubmitButton(){
  const { pending } = useFormStatus();
  return(
    <>
      <button disabled={pending}>Submit</button>
    </>
  );
}
function AddStudent(){
  return(
    <>
      <form action={async ()=>{
        await fetch("apiUrl");
        // code which may take time
      }}>
        <input type="text" name="arjun"/>
        <SubmitButton/>
      </form>
    </>
  );
}
```

Creating Custom Hook in React

- Sometimes, you'll wish that there was a Hook for some more specific purpose: for example, to fetch data, to keep track of whether the user is online, or to connect to a chat room.
- You might not find these Hooks in React, but you can create your own Hooks for your application's needs.
- Note: Custom hook in react must start with "**use**"
- You can then reuse the logic in any component.

```
import { useEffect, useState } from "react"


function useFetch(apiUrl) {
  const [data,setData] = useState([]);
  const [isLoading, setIsLoading] = useState(false);


  useEffect(()=>{
    fetch(apiUrl)
    .then(res=>res.json())
    .then(res=>setData(res))
  },[])

  return {data, isLoading}
}

export default useFetch
```

Darshan UNIVERSITY

“useDebugValue” Hook

- useDebugValue provides custom labels in React DevTools for custom hooks.

```
function useFetch(apiUrl) {
  … …
  useDebugValue(error)
  … …
}
```

“useCallback” Hook

- useCallback returns a memoized version of a function, preventing unnecessary re-creations on each render. This is helpful for passing stable callbacks to children or event handlers.

```react
import { useCallback, useState } from "react";
function Button({ onClick }) {
  return <button onClick={onClick}>Click</button>;
}
function Parent() {
  const [count, setCount] = useState(0);
  const handleClick = useCallback(() => setCount(c => c + 1), []);
  return <Button onClick={handleClick} />;
}
```

- handleClick remains stable across renders unless dependencies change.

“useMemo” Hook

- useMemo returns a memoized value to avoid expensive recalculation unless dependencies change.

```
import { useMemo, useState } from "react";
function FetchFakeData() {
  const [something,setSomething] = useState(0);
  const dataNew = useMemo(() => {
      let sum = 0;
      for(let i=0;i<10000000000;i++){
        sum+=i;
      }
      return sum;
  },[]);
  return (
  <div>
    <button onClick={()=>{setSomething(something+1)}}>Increment</button>
    sum = {dataNew} - something = {something}
  </div>
);
}
```

“useRef” Hook

- useRef provides a mutable ref object for persisting values across renders (e.g., accessing DOM nodes).

```
import { useRef } from "react";

function AccessDOM(){
  const ref = useRef();
  return(
    <>
      <input type="text" ref={ref}/>
      <button onClick={()=>{
        console.log(ref.current.value);
      }}>Click</button>
    </>
  );
}
```

“useTransition” Hook

- useTransition lets updates be marked as non-urgent, so React can keep the UI responsive.
- Use Cases for useTransition:
  - Large List Filtering/Search
  - Tab and Section Switching
  - Animations and UI Transitions
  - Data Fetching with Feedback
  - Complex State Updates

```react
import { useTransition, useState } from "react";
function TransitionExample() {
  const [isPending, startTransition] = useTransition();
  const [value, setValue] = useState("");
  const [data,setData] = useState([]);
  function handleChange(e) {
    setValue(e.target.value)
    startTransition(()=>{
      let tempData = []
      for(let i=0;i<10000;i++){
        tempData.push(e.target.value);
      }
      setData(tempData)
    })
  }
  return (
    <>
      <input onChange={handleChange} value={value} />
      {
        data.map(d=><li>{d}</li>)
      }
    </>
  );
}
```

“useDeferredValue” Hook

- useDeferredValue helps keep UI responsive by deferring non-urgent updates, valuable for slow renders.

```react/usedeferredvalue.jsx
import { useDeferredValue, useEffect, useState } from "react";

function SearchStudent(){
  const [data,setData] = useState([]);
  const [searchText,setSearchText] = useState("");
  const deferredSearchText = useDeferredValue(searchText);
  useEffect(()=>{
    let formattedData = []
    for(let i =0;i<10000;i++){
      formattedData.push(deferredSearchText)
    }
    setData(formattedData)
  },[deferredSearchText])
  return(
    <>
      <input type="text" onChange={e=>setSearchText(e.target.value)}/>
      <ul>
      {
        data.map((d)=>(<li>{d}</li>))
      }
      </ul>
    </>
  );
}
```

“useImperativeHandle” Hook

- useImperativeHandle is a React Hook that lets you customize the handle exposed as a ref.

```
import { useRef, useImperativeHandle } from 'react';
function MyInput({ ref, ...props }) {
  const inputRef = useRef(null);
  useImperativeHandle(ref, () => {
    return {
      focus() {  inputRef.current.focus();   }
    };
  }, []);
  return <input {...props} ref={inputRef} />;
}
function Form() {
  const ref = useRef(null);
  function handleClick() {
    ref.current.focus();
  }
  return (
    <form>
      <MyInput placeholder="Enter your name" ref={ref} />
      <button type="button" onClick={handleClick}>
        Edit
      </button>
    </form>
  );
}
```

Darshan UNIVERSITY

“useLayoutEffect” Hook

- useLayoutEffect is a version of useEffect that fires before the browser repaints the screen.
- useLayoutEffect runs after DOM updates but before painting, useful for measuring layouts or synchronizing state with DOM.
- useLayoutEffect can hurt performance. Prefer useEffect when possible.

```
import { useLayoutEffect, useRef } from "react";
function MeasureDiv() {
  const divRef = useRef();

  useLayoutEffect(() => {
    console.log(divRef.current.getBoundingClientRect());
  }, []);
  return (
    <>
      <div ref={divRef}>Measure me!</div>
    </>
  );
}
```

“useOptimistic” Hook

- useOptimistic is a React Hook that lets you show a different state while an async action is underway.
- It accepts some state as an argument and returns a copy of that state that can be different during the duration of an async action such as a network request.
- You provide a function that takes the current state and the input to the action, and returns the optimistic state to be used while the action is pending.

```
import React, { useState, useOptimistic, useTransition } from 'react';


// Simulates a slow network request
const updateLikesInDb = async (newCount) => {
  await new Promise((resolve) => setTimeout(resolve, 1000));
  if (Math.random() < 0.2) {
    throw new Error('Network error');
  }
  return newCount;
};


export default function LikeButton({ initialLikes }) {
  const [confirmedLikes, setConfirmedLikes] = useState(initialLikes);
  const [isPending, startTransition] = useTransition();


  const [optimisticLikes, addOptimisticLike] = useOptimistic(
    confirmedLikes,
    (currentLikes, optimisticUpdate) => currentLikes + optimisticUpdate
  );
  const handleLike = async () => {
    startTransition(async () => {
      addOptimisticLike(1);


      try {
        const updatedCount = await updateLikesInDb(confirmedLikes + 1);
        setConfirmedLikes(updatedCount);
      } catch (error) {
        console.error('Failed to update like count:', error);
      }
    });
  };


  return (
    <button onClick={handleLike} disabled={isPending}>
      {optimisticLikes} Likes
      {isPending && <span style={{ marginLeft: '8px' }}>(...updating)</span>}
    </button>
  );
}
```

“useReducer” Hook

- useReducer manages complex state logic with a reducer function, similar to Redux.
- We can think of useReducer as managing state with custom logic

```react/useReducer.jsx
import { useReducer } from "react";
function reducer(state, action) {
  switch (action.type) {
    case "increment":
      return { count: state.count + 1 };
    case "decrement":
      return { count: state.count - 1 };
    default:
      return state;
  }
}
function Counter() {
  const [state, dispatch] = useReducer(reducer, { count: 0 });
  return (
    <>
    {state.count}
    <button onClick={() => dispatch({ type: "increment" })}>
      Increment
    </button>
    <button onClick={() => dispatch({ type: "decrement" })}>
      Increment
    </button>
    </>
  );
}
```

“useSyncExternalState” Hook

- useSyncExternalStore is a React Hook that lets you subscribe to an external store.

TodoStore.js

```
let nextId = 0;
let todos = [{ id: nextId++, text: 'Todo #1' }];
let listeners = [];


export const todosStore = {
  addTodo() {
    todos = [...todos, { id: nextId++, text: 'Todo #' + nextId }]
    emitChange();
  },
  subscribe(listener) {
    listeners = [...listeners, listener];
    return () => {
      listeners = listeners.filter(l => l !== listener);
    };
  },
  getSnapshot() {
    return todos;
  }
};

function emitChange() {
  for (let listener of listeners) {
    listener();
  }
}
```

TodoApp.js

```
import { useSyncExternalStore } from 'react';
import { todosStore } from './TodoStore';

export default function TodosApp() {
  const todos = useSyncExternalStore(todosStore.subscribe,
                                     todosStore.getSnapshot);
  return (
    <>
      <button onClick={() => todosStore.addTodo()}>Add todo</button>
      <hr />
      <ul>
        {todos.map(todo => (
          <li key={todo.id}>{todo.text}</li>
        ))}
      </ul>
    </>
  );
}
```

Advanced Web Technologies (AWT)
2301CS511

Darshan UNIVERSITY
योग: कर्मसु कौशलम्

Thank You

Prof. Arjun V. Bala
Computer Engineering Department
Darshan Institute of Engineering & Technology, Rajkot
arjun.bala@darshan.ac.in
9624822202